생성 모델 관점의 분류: 나이브베이즈와 GDA

베이즈 정리와 생성 vs 판별 · GDA: 닫힌 형태의 가우시안 판별 · 나이브베이즈: 독립 가정과 차원의 저주

Machine Learning 1 · 3주차

한국과학영재학교 (KSA)

Week 3



2002년, Paul Graham: 스팸 폭탄 속에서 “확률로 걸러내면 되지 않을까?”

확률 없이: 사람(메일)을 세는 방식

	스팸 (300통)	정상 (700통)
'무료' **들어감**	$300 \times 0.6 = \mathbf{180}$	$700 \times 0.05 = \mathbf{35}$
'무료' 없음	120	665

- $P(\text{spam}|\text{무료}) = 180/215 \approx 0.837$ — **빈도를 센 것만으로** 같은 답.
- **확률 트리: 사후확률 계산**

이번 주차 개요

2002년 Paul Graham, “A Plan for Spam” — 손으로 짠 규칙에 쉽게 뚫리던 스팸 필터를 **단어 빈도 카운팅 + 베이즈 정리**로 대체 — **나이브베이즈**가 초기 스팸 필터의 표준이 됐다.

이번 장은 Week 2 로지스틱회귀와 **근본적으로 다른 방향**의 분류:

생성적(generative) 접근

판별적: $P(y|x)$ 를 **곧바로** 모델링
생성적: $P(x|y)$ “클래스가 데이터를 어떻게 만드나”를 먼저 배우고, **베이즈 정리로 뒤집기**

이 주차를 마치면:

- **사전확률 · 가능도 · 사후확률**로 스팸 확률 손계산
- 두 가능도 곡선의 **교차점**이 결정 경계가 되는 과정
- GDA: 평균 · 공분산의 닫힌 형태 추정, 경계가 직선인 이유
- 나이브베이즈 독립 가정이 안전한 이유와 깨지는 지점

3.1 베이즈 정리와 생성 vs 판별 **3.2** GDA: 닫힌 형태와 직선 경계 **3.3** 나이브베이즈: 독립 가정과 차원의 저주

3.1 베이즈 정리, 그리고 생성 vs 판별

판별적 접근과 정반대의 길

Week 2 로지스틱회귀는 $P(y|x)$ (입력이 주어졌을 때 정답의 확률)를 **곧바로** 모델링 — **판별적**(discriminative).

이번 장은 정반대: 각 클래스가 데이터를 어떻게 **만들어내는지** $P(x|y)$ 를 먼저 모델링하고, **베이즈 정리**로 뒤집어 $P(y|x)$ 를 얻는다 — **생성적**(generative).

$$P(y|x) = \frac{P(x|y)\,P(y)}{P(x)} \qquad \hat{y} = \arg\max_{y} P(x|y)\,P(y)$$

분류에서의 분모 생략

x 는 고정되고 y 만 비교하므로 $P(x)$ 는 **모든 클래스에 동일** — 무시하고 분자 $P(x|y)P(y)$ 만 최대화.

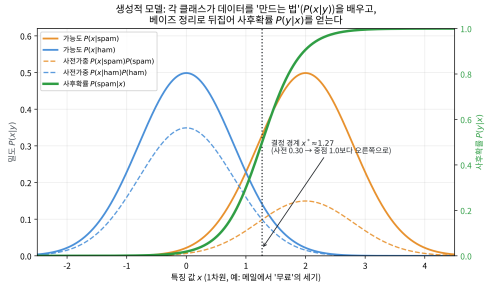
“생성적”이라는 이름의 뜻

- $P(y)$ — **사전확률**(prior): 예, 전체 메일 중 스팸 비율.
- $P(x|y)$ — **가능도**(likelihood): 예, 스팸 메일이 이런 단어들을 포함할 확률.
- $P(x|y)$**를 생성 과정으로 학습**

1차원 예: 교차점이 곧 결정 경계

특징 x = 메일에 '무료'가 얼마나 강한가.

- 각 클래스의 **가우시안 가능도**를 사전확률 (스팸 0.30)으로 가중해 겹침.
- 두 가중 곡선의 **교차점** $x^* \approx 1.27$ 이 곧 결정 경계.
- 사전이 0.5였다면 정중앙 1.0이었을 것 — **스팸이 드물다 보니** 오른쪽으로 밀림.
- 3.2절 **GDA**는 바로 이 구조의 표준형.



가중 가능도 두 곡선의 교차점 ≈1.27이 경계, 뒤집으면 사후확률 $P(\text{spam}|x)$.

왜 베이즈 “정리”가 필요한가

이 절의 모든 것이 이 한 줄에서 시작된다:

$$P(y|x) = \frac{P(x|y)P(y)}{P(x)}$$

조건부확률의 정의 $P(x|y) = \frac{P(x,y)}{P(y)}$ 에서 $P(x,y) = P(x|y)P(y)$ 로 고쳐 쓰고, 같은 $P(x,y)$ 를 반대 방향에서 다시 나누면:

$$P(y|x) = \frac{P(x,y)}{P(x)} = \frac{P(x|y)P(y)}{P(x)}$$

“정리”가 아니라 “계산”

새로운 사실이 아니라, **같은 결합확률 $P(x,y)$ 를 앞뒤로 나누는 두 방법을 이어준 등식**.

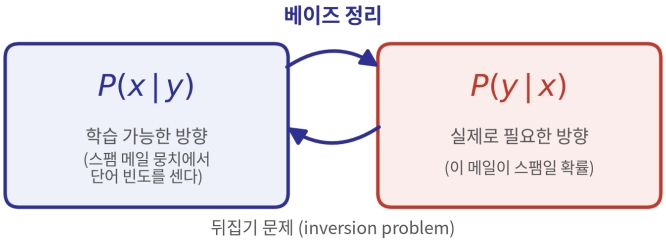
뒤집기 문제(inversion problem)

학습 가능 방향: $P(x|y)$
스팸 메일 뭉치를 꺼내 **단어 빈도를 세면** 되는 쪽 — ‘무료’가 스팸에 등장할 확률.

필요 방향: $P(y|x)$
실제 작동할 때 필요한 쪽 — ‘’무료’가 든 메일이 스팸일 확률”.

- 두 방향이 **반대**인 문제를 **뒤집기 문제**.
- 이 뒤집기를 가능하게 하는 다리는 **고유**.

그림으로: 뒤집기 문제



$$P(y|x) = \frac{P(x|y)P(y)}{P(x)}$$

학습 가능한 방향 $P(x|y)$(스팸 메일 뭉치에서 단어 빈도를 세면 됨)와 실제로 필요한 방향 $P(y|x)$(이 메일이 스팸일 확률)는 **반대** — **베이즈 정리**가 이 둘을 잇는 **고유한** 다리다.

이름의 유래: Bayes(1701–1761)

- 핵심 방식: **증거가 쌓일 때마다 확신을 갱신**.

세 단어 + 증거: 하나의 이야기

용어	뜻(스팸 예)
$P(y)$ **사전확률**	증거를 보기 **전에** 아는 비율 — 스팸 30%면 0.3
$P(x\|y)$ **가능도**	클래스 주어졌을 때 증거가 나올 확률 — '무료': 스팸 0.6, 정상 0.05
$P(y\|x)$ **사후확률**	증거를 **보았을 때** 갱신된 확률 — 분류에 쓰는 값
$P(x)$ **증거(분할합)**	어떤 클래스든 '무료'가 나올 전체 확률 — **분류에선 약수로 사라짐**

공통 어휘

이 네 단어는 Week 15 EM/GMM, VAE(ELBO = Expected Lower Bound, "사후확률의 기대 하한")까지 이어지는 **공통 어휘** — 식을 볼 때마다 "어디가 사전/가능도/사후인가"를 자동 짚을 것.

손으로 한 번: 스팸 확률 직접 계산

전체 메일의 30%가 스팸($P(\text{spam}) = 0.3$), '무료'는 스팸 60% · 정상 5%로 등장. '무료'가 든 메일의 스팸 확률:

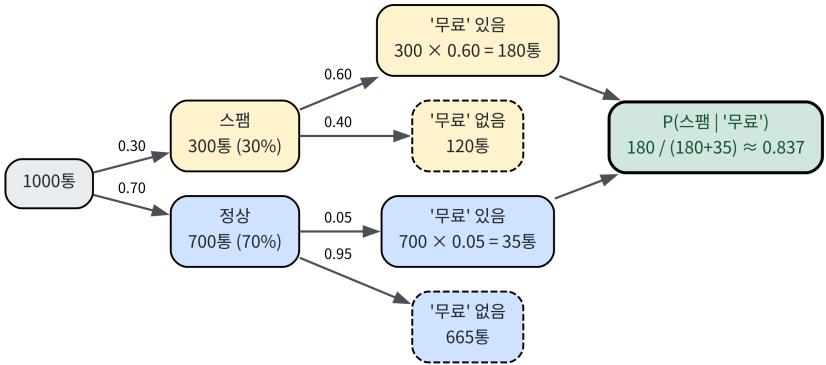
$$P(\text{spam}|\text{무료}) = \frac{0.6 \times 0.3}{0.6 \times 0.3 + 0.05 \times 0.7} = \frac{0.18}{0.18 + 0.035} = \frac{0.18}{0.215} \approx 0.837$$

하나의 단어로

사전확률 **30%** $\rightarrow$ 사후확률 $\approx$**83.7%**로 큰 도약. 이것이 나이브베이즈 필터의 본질. 3.3절에서는 이 계산을 **모든 단어**에 대해(정확히는 로그를 **더해서**) 수행.

확률 트리로 보기

1000통의 메일 — '무료' 조건부 세계(세로 합 215통) 안의 스팸 비율 180/215



'무료'가 든 215통의 조건부 세계 안에서 스팸 비율 180/215 = 사후확률.

실습: 여러 증거를 곱해서 갱신

‘무료’뿐 아니라 ‘당첨’도 등장했다면? 3.3절의 **독립 가정**을 미리 쓰면, 가능도를 **곱하기**만 하면 된다:

```
def bayes_update(prior_spam, likelihoods_spam, likelihoods_ham):
    """likelihoods_*: 관찰된각단어의   P단어클래스(|) 리스트"""
    p_spam = prior_spam
    for l_spam in likelihoods_spam:
        p_spam *= l_spam
    p_ham = 1 - prior_spam
    for l_ham in likelihoods_ham:
        p_ham *= l_ham
    return p_spam / (p_spam + p_ham)

# 무료""(0.6 vs 0.05)와 당첨""(0.4 vs 0.02)이 함께등장
result = bayes_update(0.3, [0.6, 0.4], [0.05, 0.02])
print(round(result, 4))  # 0.9904
```

증거가 하나 더해질수록 사후확률이 “거의 확실히 스팸”으로 쏠림 — **순차적 추론**의 자연스러운 틀.

오즈(odds)로 다시 쓰기

베이즈 정리 양변을 $P(y)$ 로 나누면:

$$\frac{P(y|x)}{1-P(y|x)} = \frac{P(y)}{1-P(y)} \cdot \frac{P(x|y)}{P(x|1-y)}$$

사후오즈 = 사전오즈 $\times$ 가능도비

"가능도를 곱한다"는 말은 정확히 이 한 줄 (가능도비 = likelihood ratio, $P(x|y)/P(x|1-y)$).

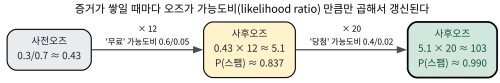
스팸 예: 사전오즈 $0.3/0.7 \approx 0.43$에서 차례로 갱신:

단계	오즈 $\rightarrow$ 확률
사전	$\approx 0.43 \rightarrow$ 0.30
+ '무료' $\times 12$	$\approx 5.1 \rightarrow$ 0.837
+ '당첨' $\times 20$	$\approx 103 \rightarrow$ 0.990

증거의 “곱의 연쇄”

- 증거가 하나씩 들어올 때마다 **오즈가 가능도비만큼만 곱**해지며 갱신.
- 이 “곱의 연쇄”가
  - 3.3절: 나이브베이즈가 **로그를 더하는** 이유
  - Week 2: 로지스틱회귀가 **시그모이드를 쓰는** 이유

  를 **같은 자리에서** 설명.



사전오즈 0.43에 가능도비 12, 20을 차례로 곱해 ≈103 (≈0.990)까지.

자주 하는 실수: 기저율 오류

가능도만 보고 사전확률 빼먹기

$$P(\text{병}|\text{양성}) = \frac{0.99 \times 0.001}{0.99 \times 0.001 + 0.01 \times 0.999} = \frac{0.00099}{0.01098} \approx \mathbf{0.090}$$

- 양성 나와도 실제로 병에 걸렸을 확률은 ≈**9%**뿐.
- **기저율 오류: 사전확률이 낮으면 사후확률도 낮음**

기저율 오류를 1000명 세기로

	병 걸림 (1명)	건강 (999명)
검사 **양성**	$1 \times 0.99 = \mathbf{0.99}$	$999 \times 0.01 = \mathbf{9.99}$
검사 음성	0.01	989.01

- **양성 10.98명 중 병든 0.99명**

유명한 사례: 튜링과 에니그마

- 튜링이 확률 갱신량의 단위를 “**반(ban)**”이라고 불렀는데, 이것은 정확히 **로그 가능도비**(log-likelihood ratio)와 같은 개념.
- 새 암호문 조각(증거)이 들어올 때마다 각 설정의 사후확률을 갱신 — 가능한 설정을 사실상 무한대 $\rightarrow$ 실용적 범위까지 축소.



앨런 튜링 (1951)
블레츨리 파크, 2차대전



독일군 **에니그마** 암호기
갱신 단위 “반(ban)” = 로그 가능도비

자주 묻는 질문 (3.1)

Q1. 생성 vs 판별, 뭐가 더 좋은가?

- **데이터 적을 때**: 생성적(GDA, 나이브베이즈) — "클래스가 데이터를 어떻게 만드나"라는 **강한 가정**을 깔고 들어가 적은 데이터로도 안정적.
- **데이터 많을 때**: 판별적(로지스틱회귀) — 가정 없이 직접 경계 학습 ⇒ 가정 틀림 위험 없음.
- 3.2절에서 GDA와 로지스틱회귀가 **같은 형태의 결정 경계**로 수렴함을 직접 확인.
- 더 깊이: Stanford CS229 노트.

Q2. $P(x)$ **를 왜 무시해도 되나?**

- 분류에서는 **고정된** x **하나**를 놓고 "$y = 0$ 이 맞을까, $y = 1$ 이 맞을까"만 비교.
- $P(x)$ 는 어떤 클래스를 비교하든 **항상 같은 값** (분모가 같으면 분자만 비교해도 대소관계 유지).
- 그래서 **분자** $P(x|y)P(y)$ **만 최대화**하는 클래스를 고르면 충분.

3.2 가우시안 판별분석 (GDA)

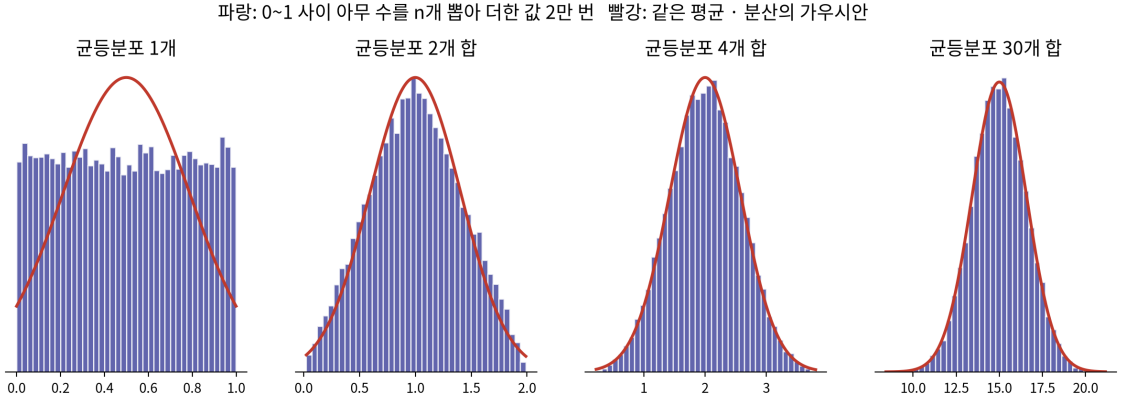
왜 가우시안인가: 가장 자연스러운 가정

- x 가 **연속**(키, 공부 시간, 픽셀)이면 — **가우시안 분포**(정규분포).

1. 평균 μ 와 공분산 Σ 두 파라미터만으로 “중심 위치”+ “퍼짐” 표현 — **간결**.
2. 현실의 연속 측정값은 수많은 작은 효과의 합 — **중심극한정리**로 가우시안에 가까워지는 경향.

각 클래스를 가우시안으로 모델링하는 분류기 = **GDA**.

그림으로: 중심극한정리



0과 1 사이 아무 수를 하나 뽑으면 모든 값이 똑같이 나온다(균등). 두 개를 뽑아 **더하면** 합이 1 근처인 경우가 가장 많고 0이나 2 근처는 드물다(삼각형) — 극단값이 나오려면 둘 다 극단이어야 하니까. 더하는 개수가 늘수록 빨간 가우시안 곡선과 거의 겹친다. **중심극한정리**: 원래 분포가 무엇이든, 독립적인 값을 많이 더하면 그 합은 가우시안에 가까워진다 — 키·측정 오차처럼 **작은 효과의 합**이 종 모양인 이유.

이진 GDA의 모델: 공유 공분산

$$y \sim \text{Bernoulli}(\phi), \qquad x \mid y{=}0 \sim \mathcal{N}(\mu_0, \Sigma), \qquad x \mid y{=}1 \sim \mathcal{N}(\mu_1, \Sigma)$$

- 두 클래스가 **같은 공분산 행렬** Σ **를 공유**하되 평균 μ_0, μ_1 만 다름.
- 파라미터 $\phi, \mu_0, \mu_1, \Sigma$ 는 학습 데이터의 **평균·공분산을 그대로 계산**해서 구함 (최대우도추정, MLE).

파라미터 추정: MLE가 왜 단순해지는가

최대우도추정(MLE) — 데이터를 "가장 그럴듯하게" 설명하는 파라미터. 데이터 독립이면:

$$\ell = \sum_{i:\, y_i=0} \log P(x_i|y{=}0) + \sum_{i:\, y_i=1} \log P(x_i|y_i{=}1) + \sum_i \left[y_i \log \phi + (1-y_i) \log(1-\phi) \right]$$

- $\hat{\phi}$ = 학습 데이터 중 $y{=}1$ 의 **비율** (사전확률 추정).
- $\hat{\mu}_0, \hat{\mu}_1$ = 클래스별 **샘플 평균**.
- $\hat{\Sigma}$ = 두 클래스의 **클래스 평균 기준 편차를 함께 모아**(pooling) 제곱평균:

$$\hat{\Sigma} = \frac{1}{N} \sum_{i=1}^{N} (x_i - \hat{\mu}_{y_i})(x_i - \hat{\mu}_{y_i})^\top$$

닫힌 형태의 의미: 대입하는 순간 끝

- $\hat{\mu}_{y_i}$ = 샘플 i 가 속한 **클래스의 평균**.
- 분모가 클래스별 n_0, n_1 이 아니라 **전체** $N = n_0 + n_1$ — 바로 두 클래스의 편차를 **합쳐서** 평균내기.
- **GDA: 수식에 대입하는 순간 파라미터 끝**

손으로 한 번: 1차원 GDA로 결정 경계

공부 시간으로 시험 합격/불합격 예측 — 불합격 $[2, 3, 4]$, 합격 $[6, 7, 8]$ (크기 같아 $\phi = 0.5$).

$$\mu_0 = \frac{2+3+4}{3} = 3, \qquad \mu_1 = \frac{6+7+8}{3} = 7$$

공유 분산 — 두 그룹의 **클래스 평균 기준 편차 6개를 함께** 제곱평균:

$$\sigma^2 = \frac{1+0+1+1+0+1}{6} = \frac{2}{3} \approx 0.667$$

사전확률이 50:50이면

결정 경계(사후확률이 같아지는 지점) = 두 평균의 **정확한 중점**:

$$x^* = \frac{\mu_0 + \mu_1}{2} = \frac{3+7}{2} = \mathbf{5}$$

$x^* = 5$ 를 밀도로 검증

x	$P(x\|y{=}0)$ 에 비례	$P(x\|y{=}1)$ 에 비례	예측
4.5	0.0452	0.0022	0 (불합격)
5.0	0.0122	0.0122	경계 (동점)
5.5	0.0022	0.0452	1 (합격)

- 정확히 $x{=}5$ 를 기준으로 **예측이 뒤집힘** — 손으로 구한 $x^* = 5$ 와 정확히 일치.

결정 경계의 모양: 왜 항상 직선인가

로그 우도비로 경계 규칙

$$z(x) = \log \frac{P(y{=}1|x)}{P(y{=}0|x)} = -\tfrac{1}{2}(x-\mu_1)^\top \Sigma^{-1}(x-\mu_1) + \tfrac{1}{2}(x-\mu_0)^\top \Sigma^{-1}(x-\mu_0) + \log \tfrac{\phi}{1-\phi}$$

핵심 한 줄

$(x-\mu)^\top \Sigma^{-1}(x-\mu)$ 를 전개하면 $x^\top \Sigma^{-1} x$ 항이 **두 클래스 모두**에 등장하는데, Σ 를 공유하므로 이 **이차항이 정확히 상쇄**. 남는 것은 x 에 대한 **일차식**뿐.

직선의 w 와 b

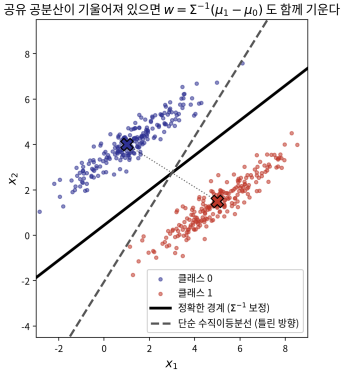
$$z(x) = w^\top x + b$$

$$w = \Sigma^{-1}(\mu_1 - \mu_0), \qquad b = -\tfrac{1}{2}(\mu_1 - \mu_0)^\top \Sigma^{-1}(\mu_1 + \mu_0) + \log \tfrac{\phi}{1-\phi}$$

결정 경계 $z(x) = 0$ 은 **직선**(d 차원이면 초평면) — "**선형** 판별"이라는 이름이 여기서 옴.

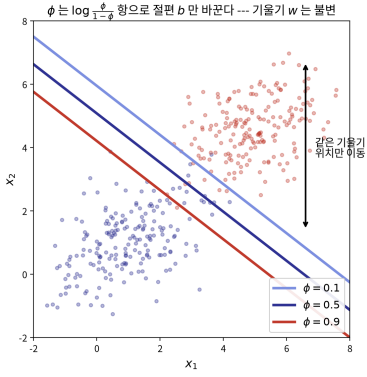
w 의 방향: 마할라노비스 보정

- w **방향: 마할라노비스 거리로 두 평균을 잇는 방향**
- Σ^{-1} 이 데이터의 **퍼진 모양에 맞게** 방향을 보정.
- 단순 수직이등분선(점선)과는 **다른 방향**.



ϕ 는 위치만, 기울기는 그대로

- **사전확률 ϕ 는 방향이 아니라 위치(b)만 움직인다.**
- ϕ 는 $\log \frac{\phi}{1-\phi}$ 항에만 등장 — 클래스 비율이 무엇이든 **직선의 기울기는 그대로**, **밀리는 정도**만 바뀜.



로지스틱회귀와 만나는 지점

놀라운 사실 (Ng & Jordan, 2001)

이렇게 구한 $P(y{=}1|x)$ 를 베이즈 정리로 전개하면, 정확히 Week 2 로지스틱회귀의 **같은 시그모이드 형태**

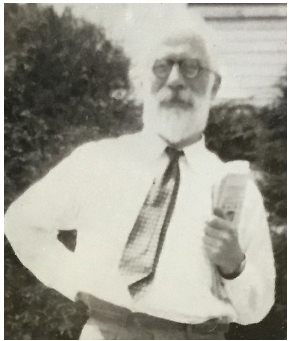
$$P(y{=}1|x) = \sigma(w^\top x + b)$$

가 나온다.

- GDA = "로지스틱회귀와 같은 결정 경계(직선)에 도달하는 **또 다른 길**".
- "모델을 데이터에 맞출 것인가, 데이터가 모델을 따르는가" — 생성적 vs 판별적의 근본적 갈림.

유명한 사례: 피셔의 1936년 선형 판별

- **꽃을 단일 숫자 $z = w^\top x$ 로 꾀서 최적 w 찾기**
- **최적 방향:** $w \propto S_W^{-1}(\mu_1 - \mu_0)$
- **피셔 LDA와 GDA는 동일한 모델**



로널드 피셔 (1952)
통계학 컨퍼런스, Asheville NC



Iris setosa



Iris versicolor

1936년 “분류 문제에서 다중 측정의 이용” — 붓꽃(iris) 종을 꽃술·꽃잎 측정값으로 분류한 데이터셋이 오늘날 **피셔 선형 판별**의 기원

실현: numpy로 GDA 구현

```python
import numpy as np

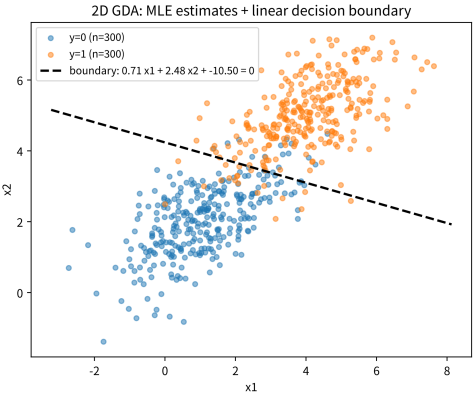
def fit_gda(X, y):
    """GDA 파라미터 (phi, mu0, mu1, Sigma)의 최대우도추정."""
    phi = y.mean()                                          # P(y=1) 추정
    mu0, mu1 = X[y == 0].mean(axis=0), X[y == 1].mean(axis=0)
    d0, d1 = X[y == 0] - mu0, X[y == 1] - mu1
    Sigma = (d0.T @ d0 + d1.T @ d1) / len(X)        # 두클래스편차   pooling
    return phi, mu0, mu1, Sigma

def gda_scores(X, phi, mu0, mu1, Sigma):
    """log P(y=1|x)/P(y=0|x) = w^Tx + b 결정 ( 경계는직선 !)"""
    Sigma_inv = np.linalg.inv(Sigma)
    w = Sigma_inv @ (mu1 - mu0)
    b = -0.5 * (mu1 - mu0) @ Sigma_inv @ (mu1 + mu0) + np.log(phi / (1 - phi))
    return X @ w + b, w, b
```

fit_gda = MLE(닫힌 형태), gda_scores = 로그 우도비 $z(x) = w^\top x + b$.

2차원 GDA 시각화

- 2차원 가우시안 300+300개 샘플로 검증: 참 평균 $[1, 2]$, $[4, 5]$, 참 공분산 $\begin{bmatrix} 1.5 & 0.8 \\ 0.8 & 1.2 \end{bmatrix}$.
- 추정 $\hat{\mu}_0, \hat{\mu}_1$ 이 참 평균에 **접근** (300개 표본이라 노이즈 ± 0.1 정도).
- 결정 경계 = $w_0 x_1 + w_1 x_2 + b = 0$ 인 **직선**.
- 기울기는 공분산의 "퍼진 방향"에 맞춰 보정된 w 가 결정.



MLE로 추정한 두 클래스 가우시안과 결정 경계 직선(검은 점선).

실습: sklearn LDA로 GDA 확인

scikit-learn의 **LinearDiscriminantAnalysis**(LDA)가 정확히 "공분산을 공유하는" GDA에 해당 — 손 계산 예로 확인:

```
import numpy as np
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

X = np.array([[2.0], [3.0], [4.0], [6.0], [7.0], [8.0]])
y = np.array([0, 0, 0, 1, 1, 1])

lda = LinearDiscriminantAnalysis()
lda.fit(X, y)
print(lda.predict([[4.5], [5.0], [5.5]]))   # [0 0 1] (5.0 근처가경계 )
print(lda.means_)                            # [[3.] [7.]]
```

$lda.means_ = [3, 7]$ — 손으로 구한 $\mu_0 = 3, \mu_1 = 7$ 과 정확히 일치. **실제 predict 출력은** $[0, 0, 1]$ — $x = 5.0$ 은 경계 위 동점이므로 sklearn은 "동점이면 클래스 0" 규칙 적용. numpy GDA와 sklearn LDA를 2차원에서 함께 돌리면 w 방향이 동일하고 예측도 거의 모든 점에서 일치(확률값만 스케일 상수 차이).

가정이 깨질 때: 불균형 사전확률

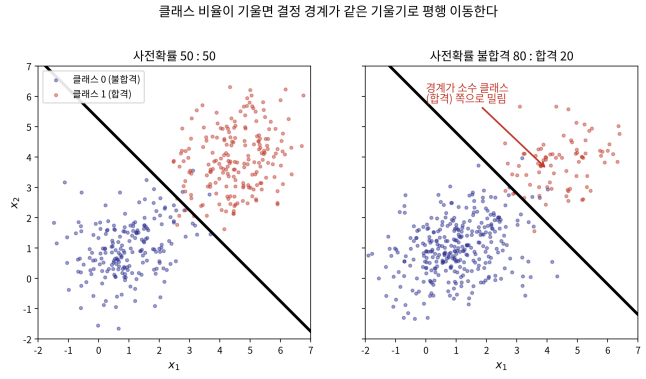
$$z(x) = 6x - 30 + \log\tfrac{0.2}{0.8}$$

경계 $z(x) = 0$

$$x^* = \frac{30 + \log(0.8/0.2)}{6} = \frac{30 + 1.386}{6} \approx \mathbf{5.23}$$

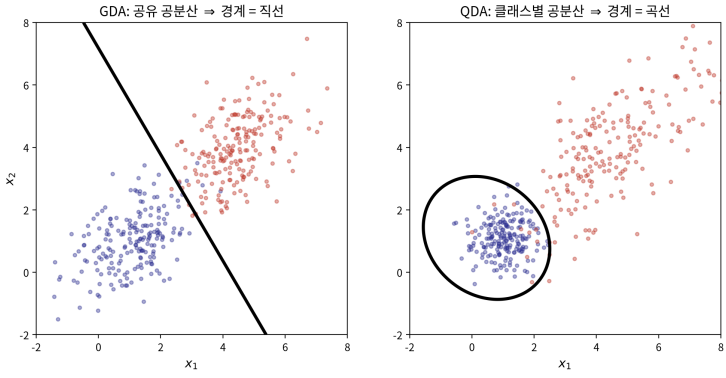
50:50일 때 경계 $x^* = 5$ 였는데, 불합격 80%로 **경계 5.23으로 밀림 — 소수 클래스(합격) 쪽으로**.

경계의 물리적 이동 = 기저율 오류



클래스 비율이 50:50에서 80:20으로 기울면, **같은 기울기**의 경계가 소수 클래스 쪽으로 밀려 분류기가 소수 클래스에 대해 **보수적**이 된다.

자주 하는 실수: 공유 공분산 가정



GDA는 두 클래스가 같은 공분산으로 퍼져 있다고 가정해 경계가 **직선**이 되지만, 실제로 공분산이 다르면 **QDA**(클래스마다 다른 공분산 허용)처럼 경계가 **곡선**이어야 더 정확하다.

자주 묻는 질문 (3.2) I

- **Q. Σ 공유가 항상 나쁜가?** 아니다 — **데이터 적을 때 장점**: 공분산 행렬을 공유하면 추정 파라미터가 절반으로 줄어(QDA보다) 적은 데이터로도 안정적. 데이터가 많고 실제로 모양이 다르면 QDA가 더 정확할 수 있음.
- **Q. 로지스틱회귀와 같은 경계로 수렴하는데 왜 GDA를?** 데이터가 정규분포에 가깝고 **표본이 적을 때**, "정규분포" 라는 강한 사전 지식으로 **적은 데이터로도** 빠르게 안정적. 로지스틱회귀는 가정이 없으므로 분포를 따르지 않아도 안정적 — 3.1 FAQ의 생성 vs 판별 트레이드오프가 실전 선택 기준.

자주 묻는 질문 (3.2) II

- **Q. $N \leq d$ 일 때 $\hat{\Sigma}$ 의 역전렬이 안 되면?** 공분산이 비특이하려면 **표본** N **> 특징** d 가 대략 필요. $N \leq d$ (문장 수백 개, 단어 특징 수천 개)면 $\hat{\Sigma}$ 가 **특이 행렬** $\rightarrow \Sigma^{-1}$ 존재 X. 실전: **자이더**(대각 정규화) $\hat{\Sigma} \leftarrow \hat{\Sigma} + \lambda I$ — 2.1절 L2 정규화와 같은 직관: “데이터가 부족하면 샘플 통계를 100% 믿지 말고 안전한 모양으로 당긴다”. 특징 수가 매우 크면(텍스트) 공분산 행렬 자체를 피하는 **나이브베이즈**(3.3)로 갈아타는 게 보통 더 낫다.
- **Q. GDA는 이진 분류에서만?** 아니다 — K 클래스: 각 클래스에 가우시안 (μ_k, Σ) 공유를 두고 $P(x|y{=}k)P(y{=}k)$ 최댓값을 예측. 어느 두 클래스 간 경계도 여전히 **선형**(QDA는 쌍마다 다른 곡선). sklearn LDA/QDA가 이 옵션들을 직접 지원.

3.3 나이브베이즈: 독립을 가정하고 차원의 저주를 피한다

독립 가정으로 차원의 저주를 피하다

나이브베이즈는 과감한 단순화로 회피 — **클래스 y 가 주어지면 각 특징(단어)이 서로 독립**:

$$P(x|y) = \prod_{j=1}^{n} P(x_j|y)$$

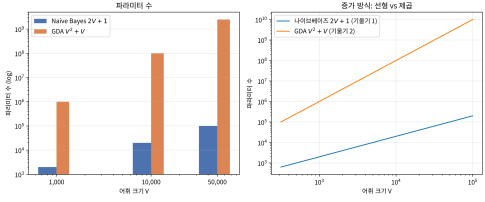
- **파라미터가 어휘 크기에 선형으로 증가**

파라미터 수: 나이브베이즈가 이기는 자리

V	나이브베이즈 $2V+1$	GDA V^2+V	격차
1,000	2,001	1,001,000	약 500배
10,000	20,001	100,010,000	약 5,000배
50,000	100,001	2,500,050,000	약 25,000배

NB의 $+1$ = 사전확률 ϕ 하나. GDA의 $+V$ = $\mu_1 - \mu_0$ 차 벡터 V개, V^2 = 공분산 행렬 (3.2절의 $\mu_0, \mu_1, \Sigma, \phi$ 바로 그 파라미터).

log-log에서 두 직선의 기울기 1 vs 2 — **"선형 vs 제곱"** 증가.



선형 vs 제곱.

왜 “증가 방식”이 품질을 좌우하나

- 나이브베이즈는 Σ 의 **모든 옳각성분을 0로 선언**(= 독립 가정)으로 원천 회피.
- **정확히는 틀린 가정**이지만, “틀려도 안전한 방향” — 상관 전부를 무시하고 **대각선만 정확히 세는** 쪽이 차원이 크면 거의 항상 이득.

의존성을 무시해도 되는 이유: 과대평가의 방향

- 나이브베이즈는 가능도를 $P(\text{무료}|\text{spam}) \times P(\text{당첨}|\text{spam})$ 로 곱해 — 실제로는 그보다 **더 큰 값**이 될 가능성을 무시한 채 계산.
- 즉, 독립 가정은 “같은 클래스 안의 증거가 쌓이면 쌓을수록” 가능도를 **과대평가**하는 **편향**(bias).
- 두 클래스의 점수가 **비슷한 방향으로** 왜곡되면 **순위**는 거의 안 바뀌고 — “둘 다 10%씩 부풀렸을 때 누가 이기나”는 원래 순서와 대개 동일.

안전장치가 항상 작동하지는 않는다

의존성의 방향이 두 클래스에서 다르면 순위가 뒤집힐 수 있다.

- 정확한 이해: “나이브베이즈가 **영원히** 틀리지 않는다”가 아니라, **틀리더라도 대개 동일한 방향으로 틀린다**.

간단한 역사: 스펠링 수정기에서 스팸 필터까지

- 1980년대 후반∼1990년대: **Peter Norvig**와 Peter Fellbaum 등 — **자동 스펠링 수정**.
- **2002년 Paul Graham 스팸 필터 대중화**
- **나이브베이즈는 텍스트 분류 기본 baseline**



Peter Norvig (2000)
1980~90년대, 나이브베이즈로 자동 스펠링 수정



Paul Graham, Y Combinator (2009)
2002년 “A Plan for Spam”으로 나이브베이즈 스팸 필터 대중화

다항 vs 베르누이: 독립 가정의 두 버전

베르누이 나이브베이즈

- $x_j \in \{0, 1\}$ — “단어 j 가 **등장했는가**”
- 문서 **길이와 무관** (‘free’ 1번이든 10번이든 “등장 1건”)
- 길이가 신호인 문제(짧은 제목 vs 긴 본문)에 유용.

다항(Multinomial) 나이브베이즈

- x_j = “단어 j 가 **몇 번** 나왔는가”
- $P(x|y) = \prod_j P(x_j|y)^{x_j}$ (다항분포)
- ‘free’ 10번 메일 $>$ 1번 메일처럼 보고 싶을 때 (대부분의 스팸/감성 분석) **자연스러움**.

공통

둘 다 “클래스가 주어지면 단어들이 독립” + **빈도 추정**의 같은 구조. 실전에선 보통 다항이 더 잘 됨 (scikit-learn `MultinomialNB`). 이번 절 코드는 **베르누이** (교육적으로 단순).

실습: 나이브베이즈 스팸 필터 (베르누이)

```python
import math

def train_naive_bayes(emails, labels):
    # emails: 단어리스트의리스트  , labels: 정상0()스팸/1()
    vocab = set(w for email in emails for w in email)
    n_spam = sum(1 for l in labels if l == 1)
    n_ham = len(labels) - n_spam
    word_counts = {0: {}, 1: {}}
    for email, label in zip(emails, labels):
        for w in set(email):  # 등장여부만세는   Bernoulli 나이브베이즈
            word_counts[label][w] = word_counts[label].get(w, 0) + 1
    return {"vocab": vocab, "word_counts": word_counts,
            "n_spam": n_spam, "n_ham": n_ham, "n_total": len(labels)}

def classify(email, model):
    words = set(email)
    log_prob = {}
    for label, n_docs in [(0, model["n_ham"]), (1, model["n_spam"])]:
        log_p = math.log(n_docs / model["n_total"])   # log P(y)
        for w in model["vocab"]:
            p_present = (model["word_counts"][label].get(w, 0) + 1) / (n_docs + 2)
            log_p += math.log(p_present) if w in words else math.log(1 - p_present)
        log_prob[label] = log_p
    return 1 if log_prob[1] > log_prob[0] else 0
```

라플라스 스무딩 $(+1)/(+2)$ 적용, 모든 단어의 log $P(x_j|y)$ 를 **더함**.

다항 버전: 8+8 학습, 4통 테스트

```python
import math

def train_multinomial_nb(emails, labels):
    vocab = sorted(set(w for e in emails for w in e))
    counts, nwords, ndocs = {0: {}, 1: {}}, {0: 0, 1: 0}, {0: 0, 1: 0}
    for e, l in zip(emails, labels):
        ndocs[l] += 1
        for w in e:                          # 이번엔등장횟수까지센다
            counts[l][w] = counts[l].get(w, 0) + 1
            nwords[l] += 1
    return {"vocab": vocab, "counts": counts,
            "nwords": nwords, "ndocs": ndocs, "n_total": len(labels)}

def multinomial_log_probs(email, model):
    out = {}
    for l in (0, 1):
        lp = math.log(model["ndocs"][l] / model["n_total"])
        for w in email:
            p = (model["counts"][l].get(w, 0) + 1) / \
                (model["nwords"][l] + len(model["vocab"]))   # 라플라스
            lp += math.log(p)
        out[l] = lp
    return out
```

4/4 모두 올바르게

```
spam_train = ["free money now", "win prize today", "free cash click",
              "prize free offer", "money win now", "free click here",
              "prize cash deal", "offer win cheap"]
ham_train = ["meeting project tomorrow", "project deadline review",
             "team meeting report", "meeting notes call", "project update plan",
             "report review friday", "team lunch call", "meeting agenda notes"]

emails = spam_train + ham_train
labels = [1]*8 + [0]*8
model = train_multinomial_nb(emails, labels)

test_emails = [
    ("free prize money", 1),         # 학습에서보지못한스팸조합
    ("win click offer", 1),
    ("meeting project report", 0),  # 학습에서보지못한정상조합
    ("team call notes", 0),
]
for email, truth in test_emails:
    lp = multinomial_log_probs(email.split(), model)
    pred = 1 if lp[1] > lp[0] else 0
    print(f"{'SPAM' if truth else 'HAM ':4s} pred={pred} "
          f"margin={abs(lp[1]-lp[0]):.2f} nats  {email!r}")
```

4/4 정답: `SPAM pred=1 margin=4.09 / 3.58`, `HAM pred=0 margin=4.09 / 3.30` (nat).

margin: “자신 있는 예측”의 수치

- **margin** = 로그 점수 차이, nat 단위 — 실전적으로 **중요한 지표**.
- margin 4.09 nat = $e^{4.09} \approx 60$ 배 가능도비 — “거의 확실하게 스팸”.
- margin 0.3 nat = $e^{0.3} \approx 1.35$ 배 — “거의 동점” $\Rightarrow$ 데이터가 약간만 바뀌어도 뒤집힐 수 있음.

손으로 한 번: 2-단어 어휘 NB

어휘 = {‘공짜’, ‘회의’}. 학습: 스팸 {{공짜}, {공짜, 회의}}, 정상 {{회의}, {회의}}. 새 메일 {**공짜**} (“공짜”만 있고 “회의”는 없음):

$$P(\text{공짜 있음}|\text{spam}) = \frac{2+1}{2+2} = 0.75, \qquad P(\text{회의 없음}|\text{spam}) = 1 - \frac{1+1}{2+2} = 0.5$$

$$P(\text{공짜 있음}|\text{ham}) = \frac{0+1}{2+2} = 0.25, \qquad P(\text{회의 없음}|\text{ham}) = 1 - \frac{2+1}{2+2} = 0.25$$

정규화 전: 스팸 $0.5 \times 0.75 \times 0.5 = \mathbf{0.1875}$ vs 정상 $0.5 \times 0.25 \times 0.25 = 0.03125$.

{공짜} 의 사후확률 = 0.857

$$P(\text{spam}|\{\text{공짜}\}) = \frac{0.1875}{0.1875 + 0.03125} \approx \mathbf{0.857}$$

한 단어로 85.7%

'공짜' **단 하나**로 스팸 확률이 85.7%까지 도약 (가능도비 6:1). 3.1절의 베이즈 계산과 **완전히 같은 절차**, 다만 곱해지는 증거가 '공짜 있음' + '회의 없음' 두 개로 늘어난 것 — 실제 필터는 어휘 전체(수만 개)에 대해 이 곱을 전부 수행.

네 가지 메일: 경계의 지도

메일	$P(\text{spam}\|x)$	예측	직관
{공짜}	**0.857**	스팸	'공짜': 스팸 2/2, 정상 0/2
{공짜, 회의}	0.667	스팸	'공짜'가 스팸 쪽으로 끌고, '회의'(양쪽 1/2)가 약한 정상 신
{회의}	0.182	정상	'회의'만 있으면 정상($0.5 \times 0.25 \times 0.75 = 0.09375$)이 스팸
$\varnothing$ (없음)	0.400	정상	사전확률(0.5)과 '단어 없음' 신호(약한 스팸)가 부분 상쇄

단어 하나가 아니라 전체 메일을 본다

자주 하는 실수: 카운트 방식 선택

- “등장 **여부만** 볼 것인가(베르누이), **횟수** 까지 볼 것인가(다항), 볼 때 어떻게 정규화할 것인가”를 **의식적으로** 정하고, 데이터셋 특성(길이가 신호인가, 반복 빈도가 신호인가)에 맞게 고를 것.

자주 하는 실수: 스무딩 없이 곱하기

못 본 단어면 전체 곱이 0

- **라플라스 스무딩은 보수적 가정**

스무딩 효과 숫자로: ‘당첨’이 들어온 메일

	P(당첨 있음|spam)	P(당첨 있음|ham)
스무딩 없이	$0/2 = \mathbf{0}$	$0/2 = \mathbf{0}$ $\Rightarrow$ 0 vs 0 **비교 불가**
라플라스	$1/4$	$1/4$ $\Rightarrow$ **중립** 신호

- **스무딩으로 ’공짜’가 스팸 판단 결정**

사전확률 $P(y)$ 의 힘: 그레이엄의 "스팸이 90%"

NB 점수 첫 항 $\log P(y)$ 가 사전확률

$$P(\text{공짜}|\text{spam})\,P(\text{회의}|\text{spam}) = \tfrac{3}{4} \times \tfrac{1}{2} = \tfrac{3}{8} = \tfrac{1}{4} \times \tfrac{3}{4} = P(\text{공짜}|\text{ham})\,P(\text{회의}|\text{ham})$$

가능도비 2 $\Rightarrow$ 분류는 $P(y)$ 로 결정

가능도비 2: 사전확률이 예측을 뒤집음

학습된 스팸 비율(사전확률)	$P(\text{spam} \mid \{\text{공짜}, \text{회의}\})$	예측
50%	0.667	스팸 (**불확실하게**)
90%	0.947	스팸 (**거의 확실하게**)
10%	**0.182**	**정상** (예측 뒤집힘)

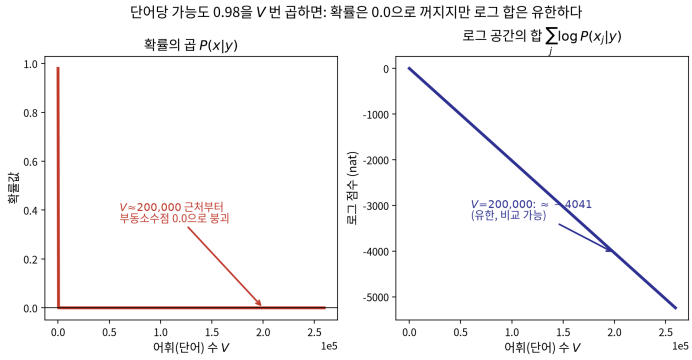
- **단어 증거 2:1 우위가 사전확률 9:1에 밀림**

확인 문제: 사전확률만으로 뒤집히는 메일

- **(a)** 같은 학습 데이터에서, 사전확률 **50/50**과 **90/10**(스팸/정상)일 때의 $P(\text{spam} \mid \{\text{회의}\})$ 를 라플라스 스무딩으로 계산 — 어느 쪽에서 '회의'만 있는 메일을 **스팸**으로 분류하게 되는가? 그 차이가 사전확률 $\log \frac{0.9}{0.1} \approx 2.2$ nat에서 오는 것인지, 가능도에서 오는 것인지 **구별**. 답: 50/50 $\rightarrow$ 0.40 (정상), 90/10 $\rightarrow \frac{0.9 \times 0.5}{0.9 \times 0.5 + 0.1 \times 0.75} \approx 0.857$ (스팸). 가능도비 2/3은 동일, 바뀐 건 **사전확률 우위 9:1**.
- **(b)** 가능도비 r 로 쓸 때: 50/50에서 정상 예측 $\Leftrightarrow r < 1$, 90/10에서 스팸 예측 $\Leftrightarrow 9r > 1 \Leftrightarrow r > 1/9$.

  $1/9 < r < 1$인 메일만 50/50 $\rightarrow$ 90/10 변화로 뒤집힘. 예: $r{=}0.5$ 면 1/3(정상) $\rightarrow$ 0.818(스팸). 반대로, $r{=}2$(스팸 쪽)인 메일은 50/50 $\rightarrow$ 10/90 방향에서만 뒤집힘. 한 문장: **가능도비가 1 근처($\sim$1/9 $\sim$ 9)인 "동점이 가까운" 메일만, 사전확률의 9:1 변화에 의해 뒤집힘**.

왜 로그를 더하는가: 언더플로우 방지



확률을 곱하면 underflow — float64는 10^{-308} 이하를 정확히 0.0으로 떨어뜨리지만, **로그 공간**에서는 합이 **항상 유한**해서 비교가 가능하다.

이 우도비, 실전 어디서 또 만나는가

- 이 **로그 우도비**로 단어를 **순위 매기면** “스팸을 가장 강하게 가리키는 단어 top-N”이 나온다.

코퍼스언어학의 “keyness”가 바로 이것

AntConc 같은 코퍼스 분석 도구의 **keyness** 측정치(Mutual Information, Log Ratio)는 “이 단어가 타깃 코퍼스에서 기준 코퍼스보다 얼마나 튀는가”를 정확히 같은 형태 $\log(O/E)$ — **관측/기댓값의 로그비**로 잰다. 스팸 필터의 “이 단어가 스팸을 얼마나 가리키는가”와 수학적으로 동일한 질문.

자주 묻는 질문 (3.3)

- **Q. 독립 가정이 명백히 틀렸는데 왜 잘 되나?** 분류는 **정확한 확률값**을 완벽히 맞힐 필요가 없고 **어느 클래스 점수가 큰가**(순위)만 맞히면 됨. 의존성 무시로 확률값이 왜곡돼도, 그 왜곡이 **두 클래스 모두에 비슷한 방향**으로 일어나는 경우가 많아서 **상대 순위**가 크게 안 틀림.
- **Q. 텍스트 데이터엔 뭘 써야 하나? 거의 항상 나이브베이즈** — 어휘가 수만이면 GDA의 Σ(어휘 크기 **제곱** 원소) 추정에 데이터가 절대 부족. 반대로 **저차원 연속값**(키·몸무게 등)이면 GDA가 특징 간 상관(공분산)까지 반영해 유리.
- 실전 고르는 기준: **"특징이 많고 이산적?** $\rightarrow$ **NB. 적고 연속?** $\rightarrow$ **GDA."** 더 깊이: Stanford CS229 노트.

3.2 + 3.3의 공통 철학

GDA(연속값에 정규분포)와 NB(이산값에 독립 가정)는 접근 방식은 달라도, **둘 다 "먼저 각 클래스가 데이터를 어떻게 만드나를 모델링하고, 베이즈 정리로 뒤집는다"는 생성적 철학**을 공유.

이번 주차 정리

1. **3.1 베이즈 정리 = "같은 $P(x, y)$ 를 앞뒤로 나누는"**
   세 단어(사전·가능도·사후) + "세로 합 = 조건부 세계" 확률 트리. **오즈 = 가능도비 곱의 연쇄**가 NB의 로그 합과 로지스틱 시그모이드를 같은 자리에서 설명. **기저율 오류**: 사전확률을 빼먹으면 99% 검진도 9% 사후확률.

2. **3.2 GDA = 가우시안 + 공유 공분산 + MLE**
   평균·공분산으로 **닫힌 형태**. 이차항 상쇄 $\Rightarrow$ **결정 경계는 항상 직선**. $P(y{=}1|x) = \sigma(w^\top x + b)$ — **로지스틱회귀와 같은 시그모이드**. ϕ 는 위치만 미는 **병렬 이동** = 경계의 기저율 오류.

3. **3.3 NB = 독립 가정 + 빈도 카운팅**
   파라미터 **선형**($2V{+}1$) vs GDA **제곱**($V^2{+}V$) $\Rightarrow$ **차원의 저주 회피**. 틀려도 **동일한 방향**으로 틀려 순위 유지. 스무딩(0 강제 방지) · **로그 합**(언더플로우) · **사전확률**(가능도비 2도 10%면 뒤집힘).

다음 주 — Week 4. kNN과 k-means

또 다른 극단: **배울 파라미터가 전부 없는** 분류기. "k개 가장 가까운 이웃을 보고 다수결" — "모델 없이 거리만으로". 이번 장의 "데이터 생성 법칙을 배운다"에서 "아무것도 배우지 않는다"로 넘어가는 것. 다만 이번 장이 예고한 **차원의 저주**는 Week 4에서 "왜 kNN이 고차원에서 무너지는가" 로 다시 나옴.